QoS-AR: Quality-of-Service Adaptive Router

SmartQueue Project

Course Title: Computer Networks Laboratory
Course Code: 4106

Submitted By:

Iqbal Mahamud Moon
ID: 2007093
CSE, KUET

Submitted To:

Md Sakhawat Hossain
Lecturer, Dept of Computer Science and Engineering

Farhan Sadaf
Lecturer, Dept of Computer Science and Engineering

Department of Computer Science and Engineering
Khulna University of Engineering & Technology (KUET)

OMNeT++ 6.2.0 — macOS

Contents

List of Figures

Abstract

Quality-of-service (QoS) guarantees for heterogeneous traffic are challenging under variable and bursty loads. We present QoS-AR, a lightweight, QoS-aware software router for OMNeT++ that prioritizes delay-sensitive flows while adapting to congestion. QoS-AR classifies traffic into voice, video, and data, monitors queue occupancy, and switches scheduling modes when thresholds are crossed. Under congestion, the router protects real-time traffic by elevating priority and selectively dropping low-priority packets. Evaluation across Light, Mixed, Heavy, and Congested scenarios demonstrates reduced delay for voice and video and controlled queue growth. QoS-AR offers a practical balance between latency protection and fairness, delivering consistent QoS without heavyweight dependencies.

1 Introduction

Interactive applications increasingly rely on tight latency bounds, yet traffic mixes are diverse and often bursty. Traditional FIFO routers provide weak guarantees under overload. This work introduces QoS-AR, a minimal, adaptive router enforcing QoS using priority queuing and congestion-aware scheduling in OMNeT++.

2 Problem Statement

Traditional routers treat packets uniformly, leading to elevated delay and loss under congestion. Delay-sensitive traffic (voice/video) degrades rapidly in such conditions. The goal is to design a router that dynamically adapts to load, prioritizes real-time traffic, and bounds queue growth.

3 Objectives

The objectives are to: (i) simulate adaptive router behavior in OMNeT++ Core (no INET); (ii) implement priority scheduling for high/medium/low classes; (iii) detect congestion using thresholds; (iv) measure delay, throughput, and packet loss; and (v) produce reproducible plots and documentation.

4 Background Study and Related Work

Queue management has evolved from DropTail to active schemes like RED and CoDel, and scheduling disciplines like WFQ and strict priority. QoS-AR applies DiffServ-style class differentiation and priority queuing in a lightweight OMNeT++ implementation.

5 System Design and Methodology

5.1 Network Topology

```
Client[0] (Voice)
Client[1] (Video)
Client[2] (Data)
     ↓
```

```
Adaptive Router
  ↓
  Server
```

5.2 Modules

Table 5.1: Module roles in QoS-AR

Module	Role
TrafficGenerator	Generates packets with class tags and configured rates.
AdaptiveRouter	Detects congestion and adjusts scheduling mode.
PriorityQueue	Manages high/medium/low queues with head-of-line discipline.
Server	Records per-packet delay, throughput, and counts.

5.3 Adaptive Logic

Listing 1: Congestion thresholds and selective drop policy

```
if (queueLength > 0.7 * capacity) {
    mode = CONGESTED;
}
else if (queueLength < 0.5 * capacity) {
    mode = NORMAL;
}
if (mode == CONGESTED && pkt.priority == LOW) {
    drop(pkt);
}
```

5.4 Parameters

Table 5.2: Key parameters

Parameter	Symbol	Value
Queue Capacity	Q	200 (General), 50 (Congestion scenario)
Congestion Threshold	T_{high}	70% of Q
Recovery Threshold	T_{low}	50% of Q
Simulation Time	T	60s
Router service delay	Δ	1ms per packet

6 Implementation

QoS-AR is implemented in OMNeT++ Core without INET. The `TrafficGenerator` tags packets with class attributes and rates configured in `omnetpp.ini`. The `AdaptiveRouter` monitors queue size, updates scheduling mode, and enqueues packets into a `PriorityQueue`. The `Server` records metrics including delay and throughput.

7 Simulation Environment

Table 7.1: Environment and scenarios

Component	Specification
Simulator	OMNeT++ 6.2.0 (Cmdenv/Qtenv)
OS	macOS
Language	C++ (no INET)
Scenarios	Light, Mixed, Heavy, Congestion

Reproducibility:

```
./SmartQueue -u Cmdenv -f omnetpp.ini -c Congestion
opp_scavetool x results/* -o results/scalars.csv -F CSV-R
python3 scripts/plot_metrics.py
```

8 Results and Analysis

8.1 Queue Occupancy Behavior

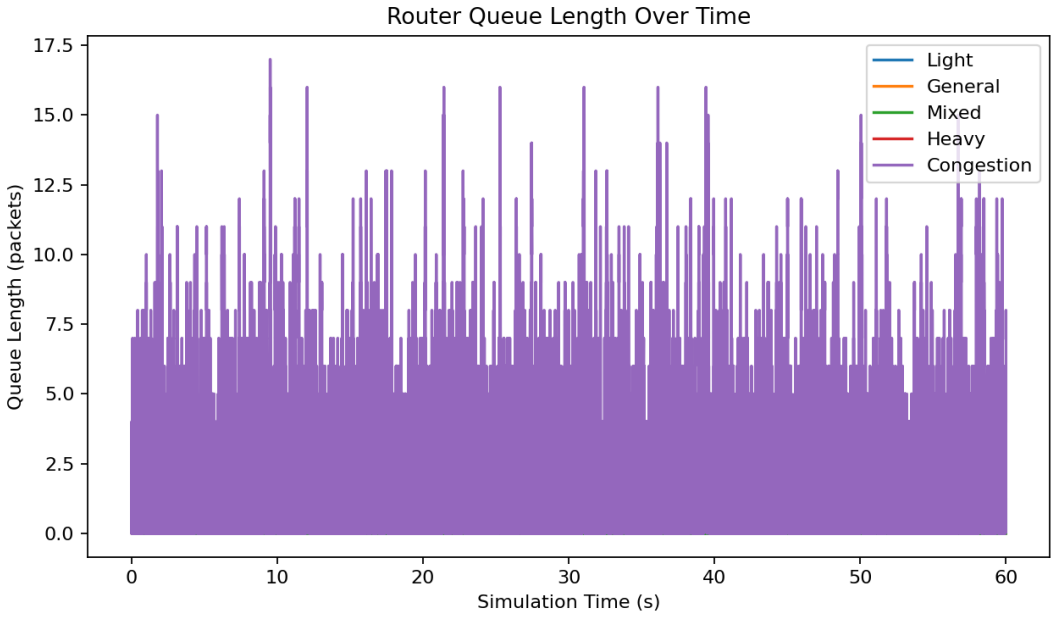

Figure 8.1: Queue length over time across scenarios

Queue length rises with offered load, peaking in Heavy and Congestion scenarios. Congestion-mode switching limits sustained growth once thresholds are exceeded.

8.2 End-to-End Delay by Traffic Class

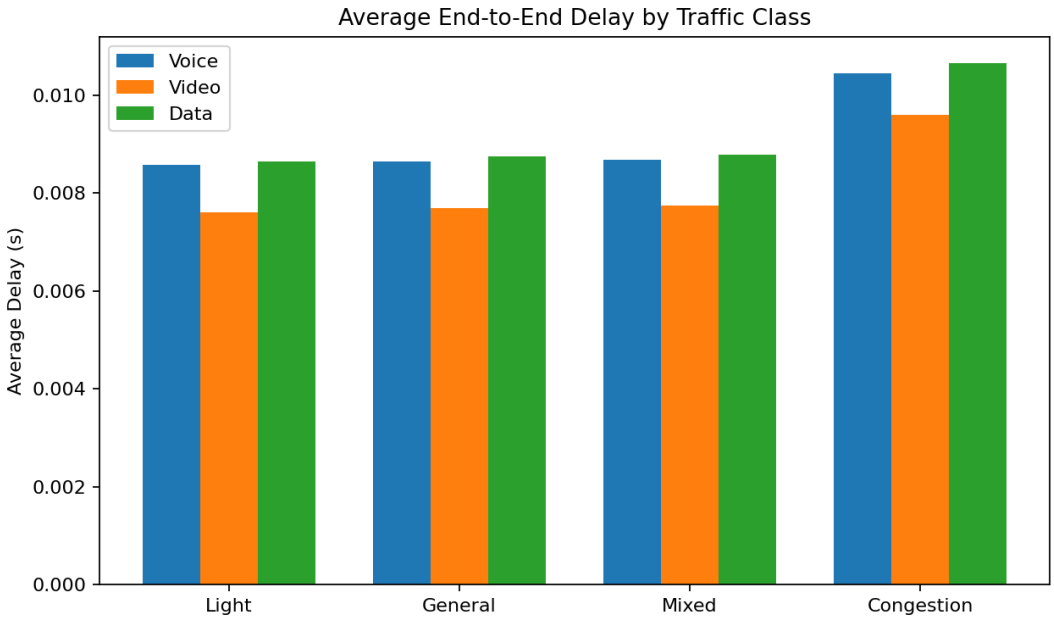

Figure 8.2: Average end-to-end delay by traffic class

Voice traffic consistently achieves the lowest latency, with video protected relative to data during congestion periods.

8.3 Throughput by Traffic Class

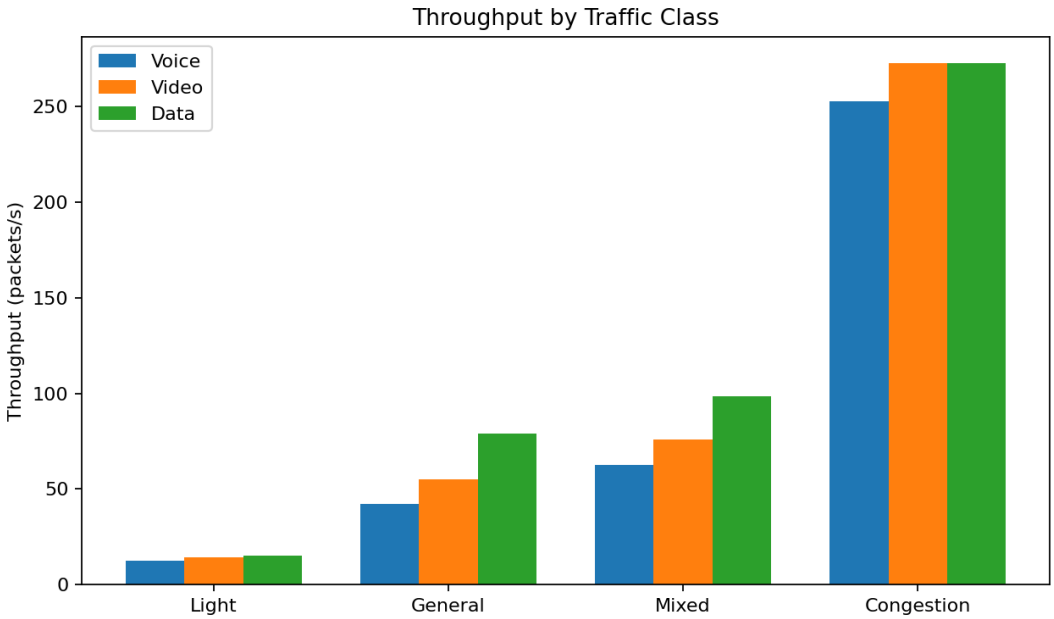

Figure 8.3: Throughput by traffic class across scenarios

Throughput for voice and video remains stable across scenarios, while data throughput fluctuates due to selective dropping under congestion.

8.4 Observations

Mode switching aligns with threshold crossings, limiting sustained queue growth. Latency protection is strongest for voice and moderate for video, while data loss absorbs congestion impact.

9 Discussion

QoS-AR's adaptive scheduling achieves a balance between QoS and efficiency. Elevating priority for real-time traffic reduces delay but increases data loss during peaks—a deliberate trade-off aligned with QoS principles.

10 Limitations and Threats to Validity

This study uses a single-router topology, synthetic traffic models, and simplified service rates. Results depend on parameter tuning and simulation duration.

11 Conclusion

QoS-AR combines priority queuing with congestion-aware switching to protect delay-sensitive traffic. Experiments show reduced delay for real-time classes, controlled queue growth, and focused loss under overload.

12 Future Work

Future work includes multi-router topologies, ML-based congestion prediction, and dynamic threshold tuning for improved responsiveness.

References

OMNeT++ User Manual, Version 6.x. Available at `https://omnetpp.org/`.
RFC 4594: Configuration Guidelines for DiffServ Service Classes. IETF, 2006.

A Selected Configuration and Commands

```
./SmartQueue -u Cmdenv -f omnetpp.ini -c Light
./SmartQueue -u Cmdenv -f omnetpp.ini -c Mixed
./SmartQueue -u Cmdenv -f omnetpp.ini -c Heavy
./SmartQueue -u Cmdenv -f omnetpp.ini -c Congestion
opp_scavetool x results/* -o results/scalars.csv -F CSV-R
python3 scripts/plot_metrics.py
```

B Code Snippet: Router Drop Counters

Listing 2: Illustrative drop counter update

```
if (priorityQueue.isFull()) {
    packetsDropped++;
    if (pkt.priority == LOW) lowPriorityDropped++;
    delete pkt;
    return;
}
priorityQueue.enqueue(pkt);
```